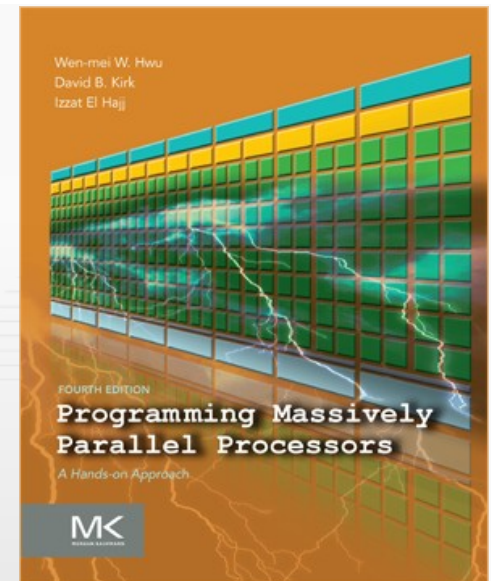


Programming Massively Parallel Processors

A Hands-on Approach

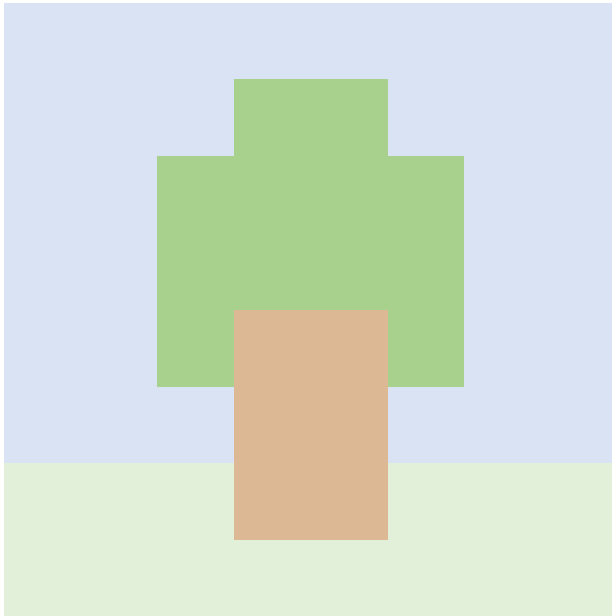
CHAPTER 3 > **Multidimensional Grids and Data**



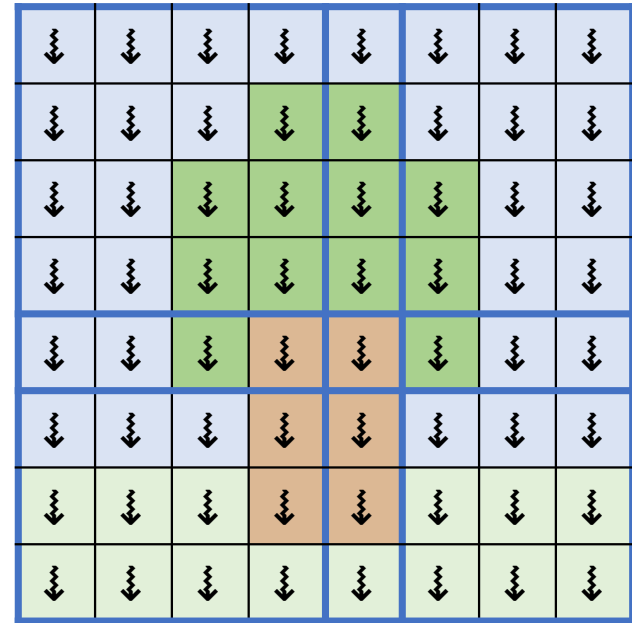


parallelization approach: assign one thread to convert each pixel in the image

- CUDA supports multidimensional grids (up to 3D)
 - Simplifies processing multidimensional data



Image

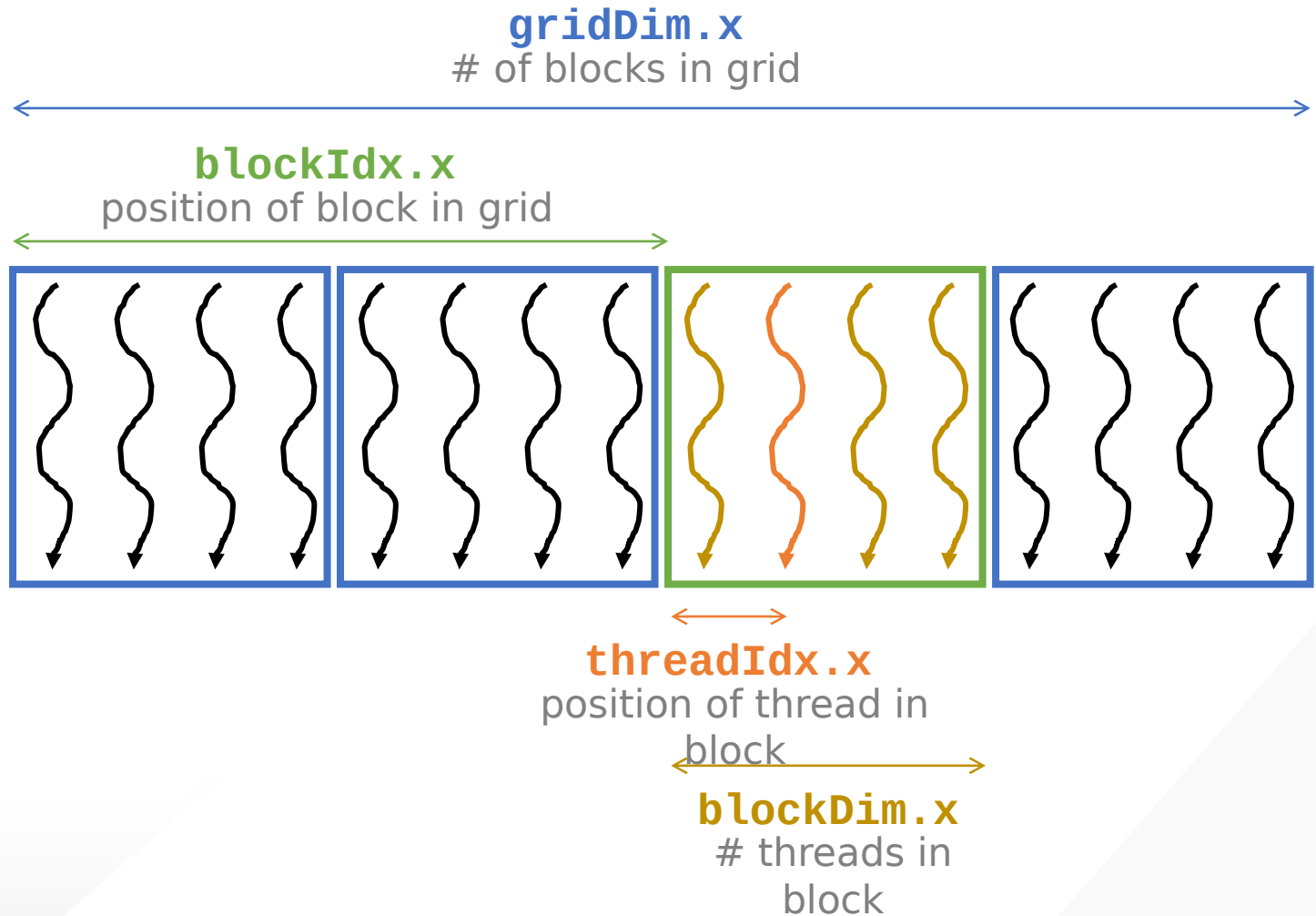


Block and Thread Assignment

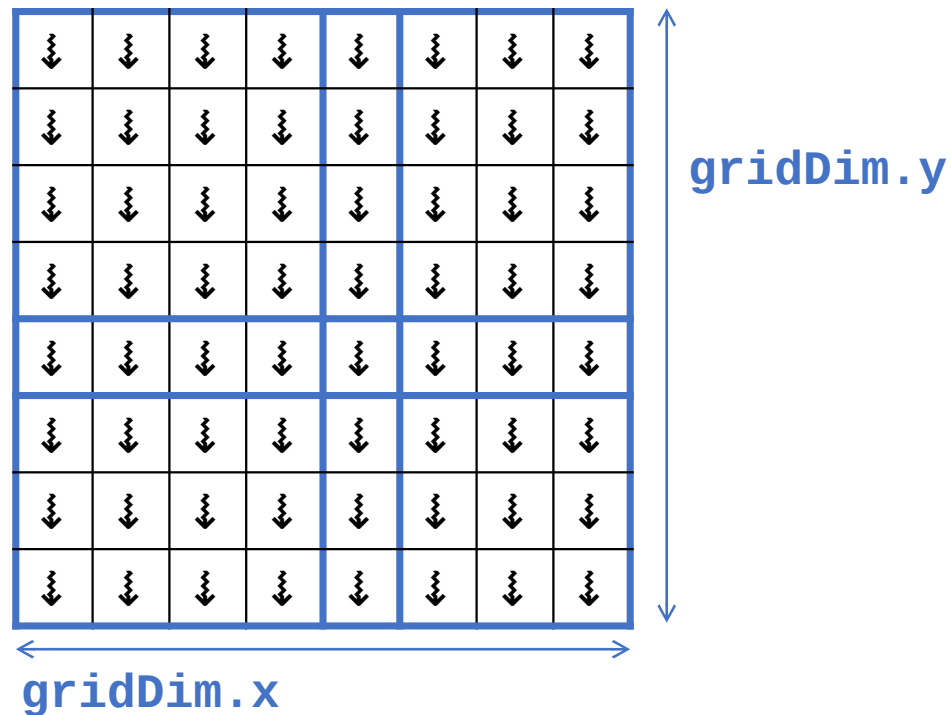
- Use built-in `dim3` type
 - The rest is the same

```
dim3 numThreadsPerBlock(32, 32);  
dim3 numBlocks((width + numThreadsPerBlock.x - 1)/numThreadsPerBlock.x,  
               (height + numThreadsPerBlock.y - 1)/numThreadsPerBlock.y);  
rgb2gray_kernel <<< numBlocks, numThreadsPerBlock >>>  
                (red_d, green_d, blue_d, gray_d, width, height);
```

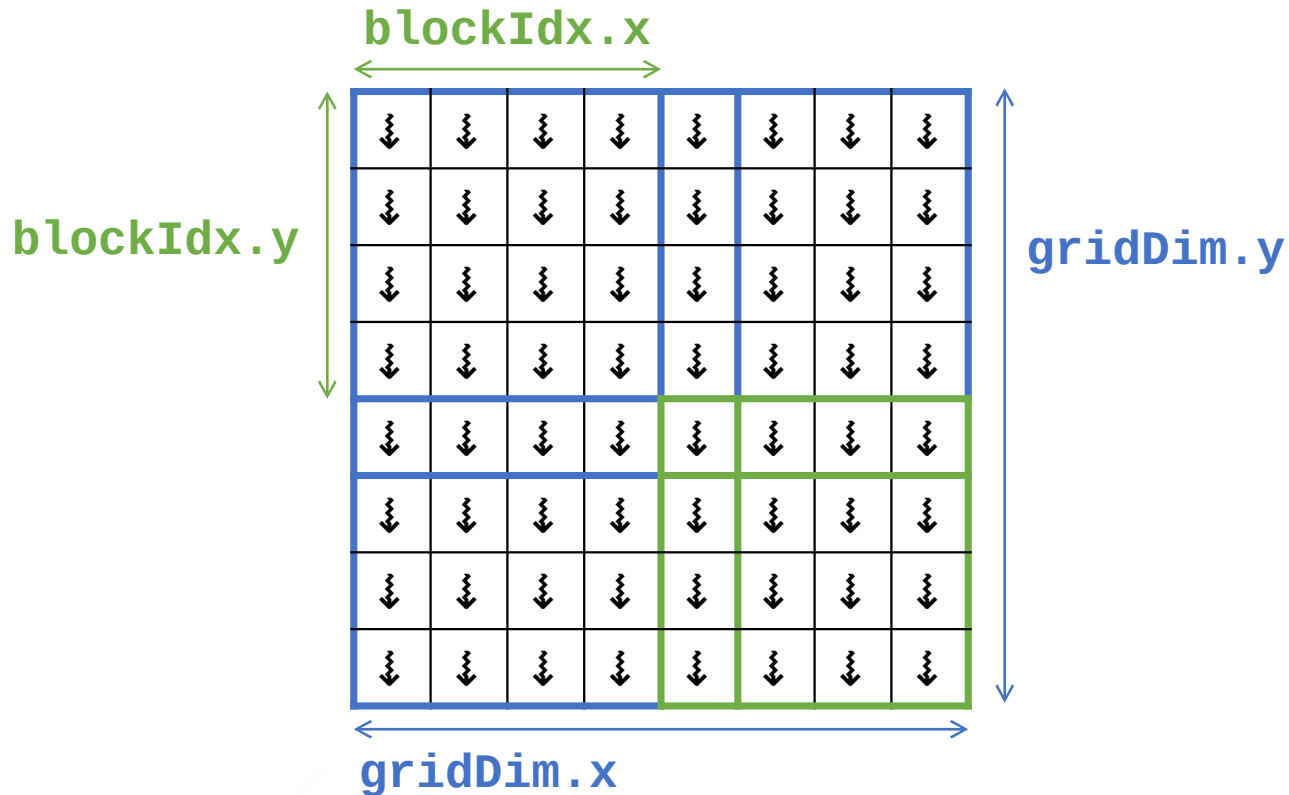
Previously: One Dimensional Indexing



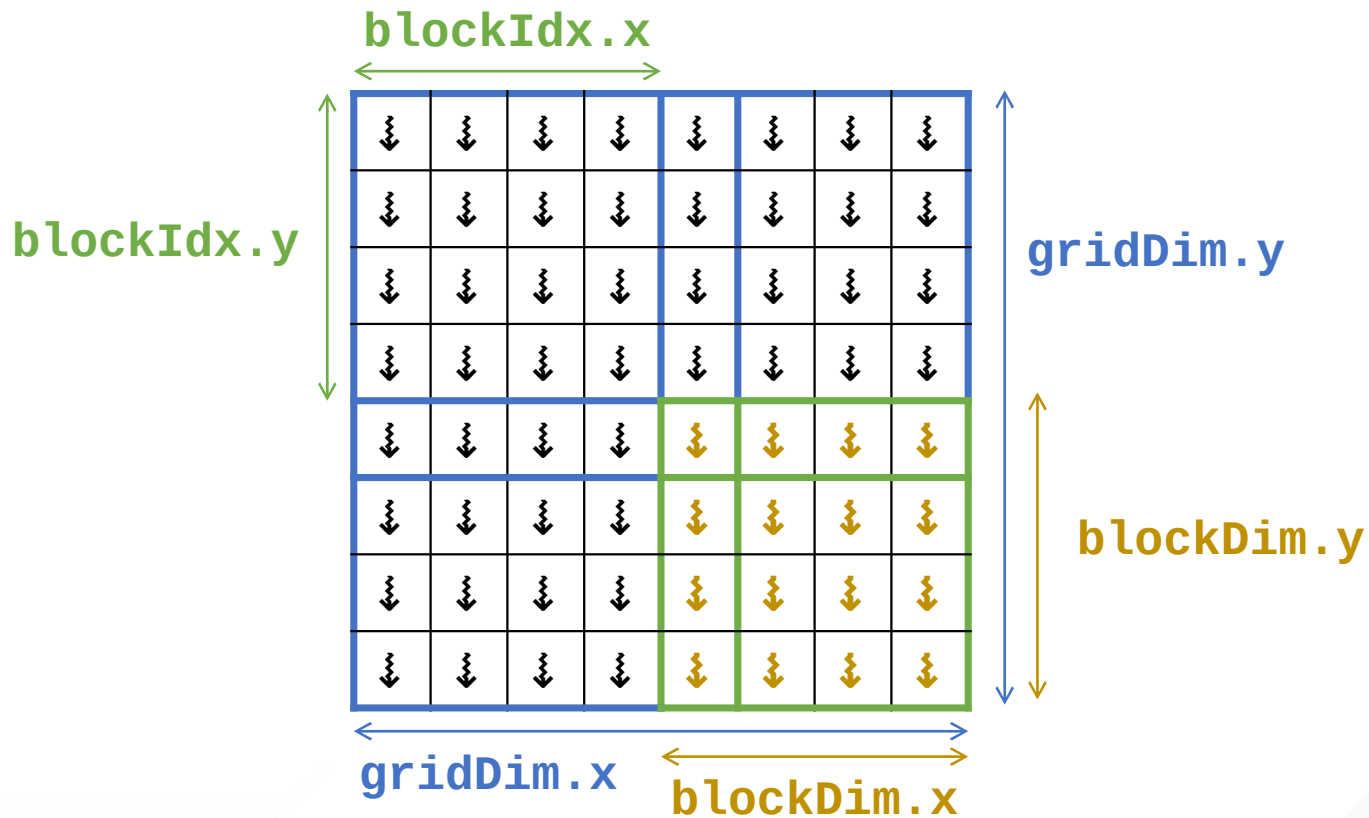
- Built-in dimension and index variables each have three components x, y, and z



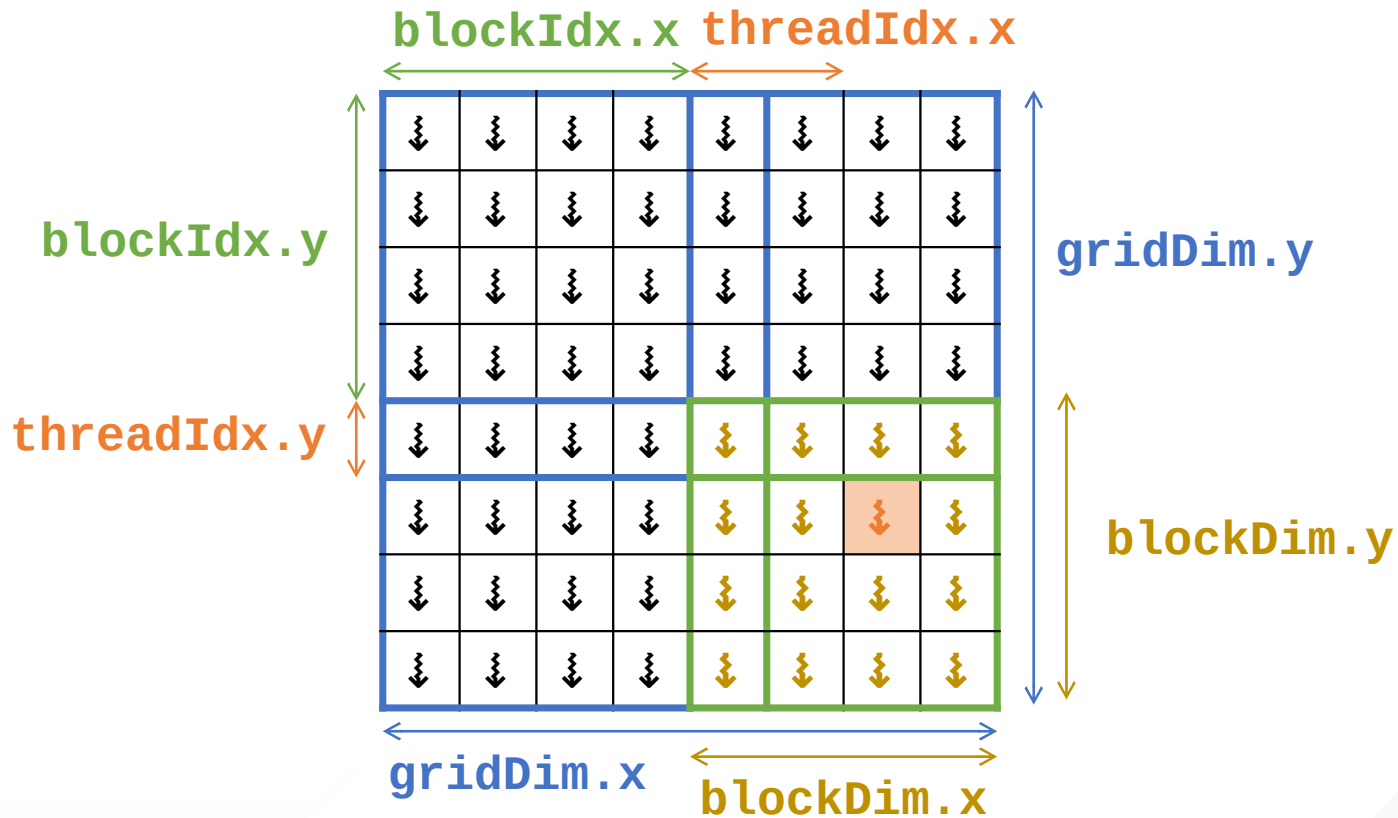
- Built-in dimension and index variables each have three components x, y, and z



- Built-in dimension and index variables each have three components x, y, and z



- Built-in dimension and index variables each have three components x, y, and z



```
__global__ void rgb2gray_kernel(unsigned char* red, unsigned char* green, unsigned char* blue,  
                                unsigned char* gray, unsigned int width, unsigned int height) {  
  
    unsigned int row = blockIdx.y*blockDim.y + threadIdx.y;  
    unsigned int col = blockIdx.x*blockDim.x + threadIdx.x;  
  
    // Convert the pixel  
    ...  
}
```

- Convention is C is to store data in **row major order**
 - Elements in the same row are contiguous in memory

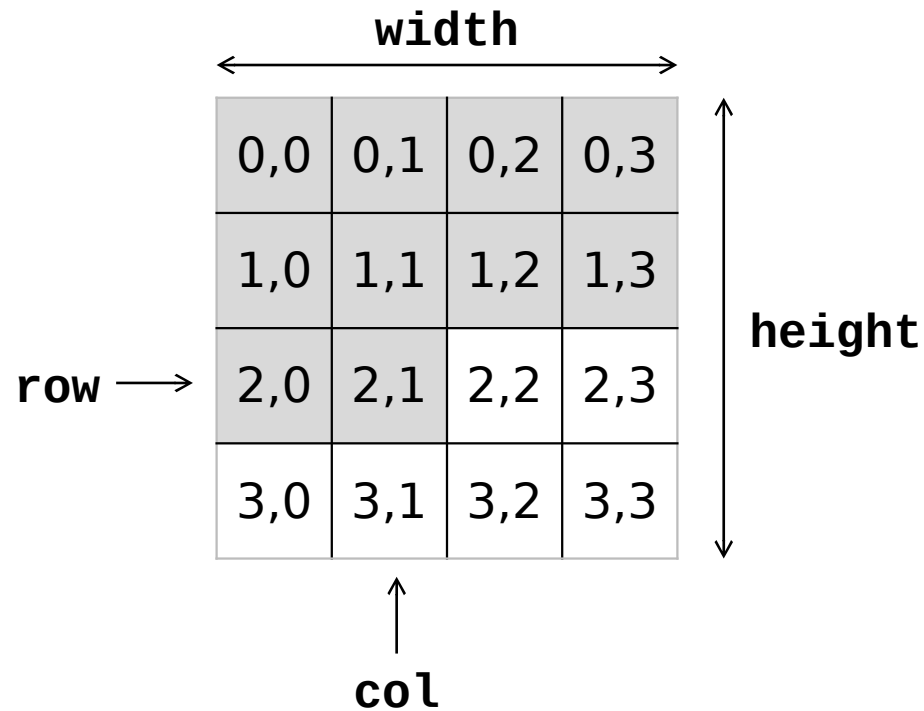
Logical view of

0,0	0,1	0,2	0,3
1,0	1,1	1,2	1,3
2,0	2,1	2,2	2,3
3,0	3,1	3,2	3,3

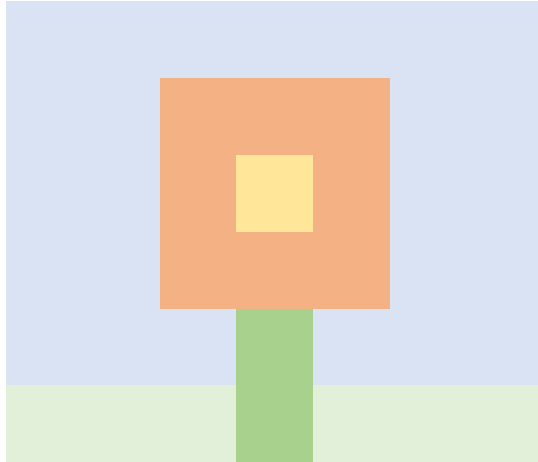
Actual layout in

0,0	0,1	0,2	0,3	1,0	1,1	1,2	1,3	2,0	2,1	2,2	2,3	3,0	3,1	3,2	3,3
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----

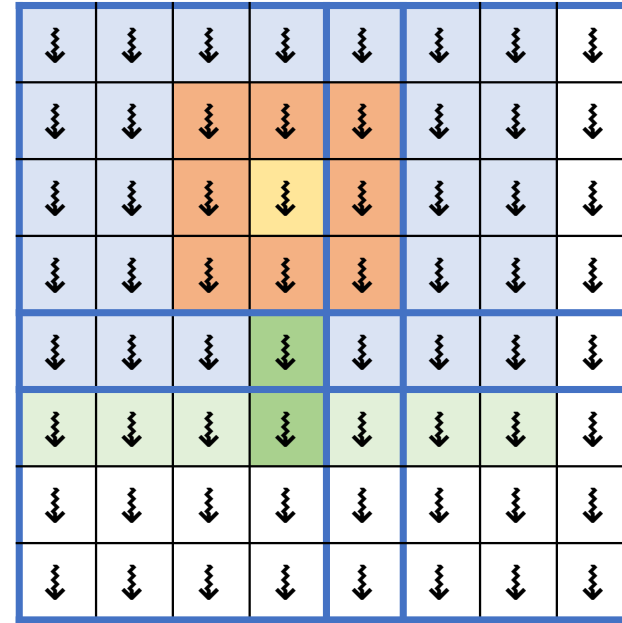
```
index = row*width + col
```



```
__global__ void rgb2gray_kernel(unsigned char* red, unsigned char* green, unsigned char* blue,  
                                unsigned char* gray, unsigned int width, unsigned int height) {  
  
    unsigned int row = blockIdx.y*blockDim.y + threadIdx.y;  
    unsigned int col = blockIdx.x*blockDim.x + threadIdx.x;  
  
    // Convert the pixel  
    gray[row*width + col] = red[row*width + col]*3/10  
        + green[row*width + col]*6/10 + blue[row*width + col]*1/10;  
  
}
```



Image



Thread Assignment

Could have more threads than pixels in both dimensions

```
__global__ void rgb2gray_kernel(unsigned char* red, unsigned char* green, unsigned char* blue,
                                unsigned char* gray, unsigned int width, unsigned int height) {
```

```
    unsigned int row = blockIdx.y*blockDim.y + threadIdx.y;
    unsigned int col = blockIdx.x*blockDim.x + threadIdx.x;
```

```
    // Convert the pixel
```

```
    if (row < height && col < width) {
        gray[row*width + col] = red[row*width + col]*3/10
            + green[row*width + col]*6/10 + blue[row*width + col]/10;
```

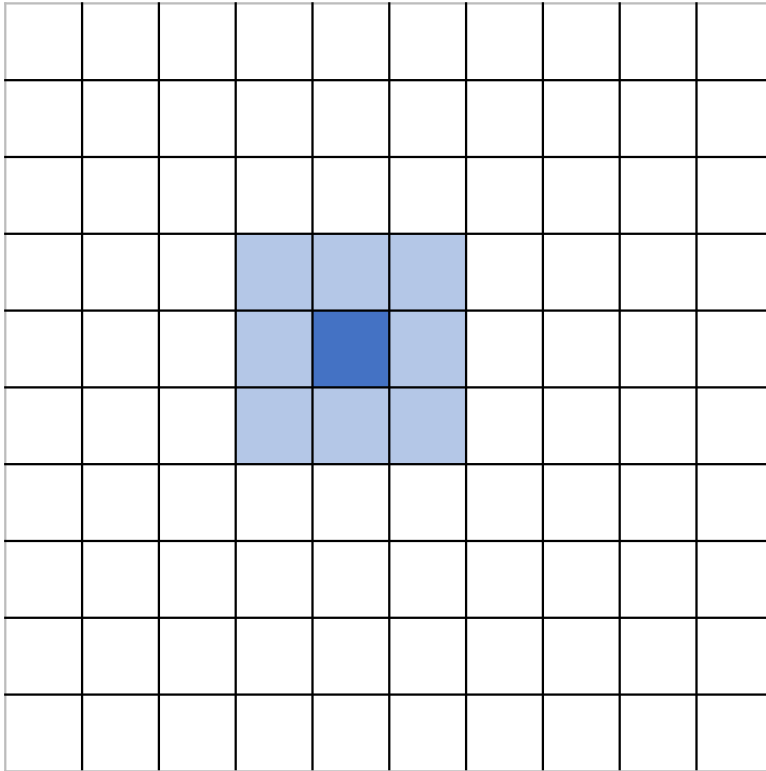
```
    }
```

```
}
```

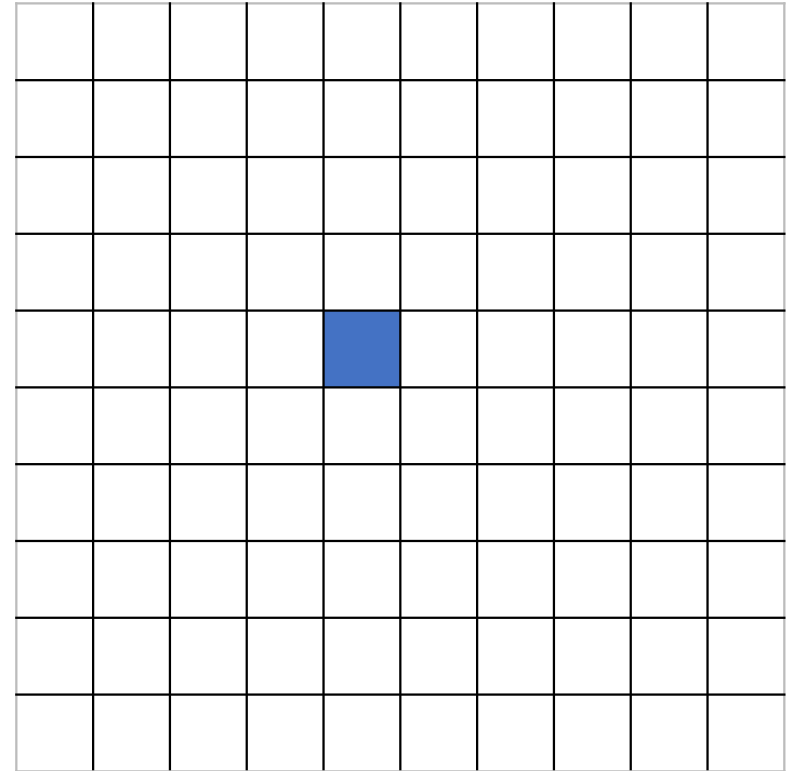
boundary check



Output pixel is the average of the corresponding input pixel and the pixels around it



Input Image



Output Blurred Image

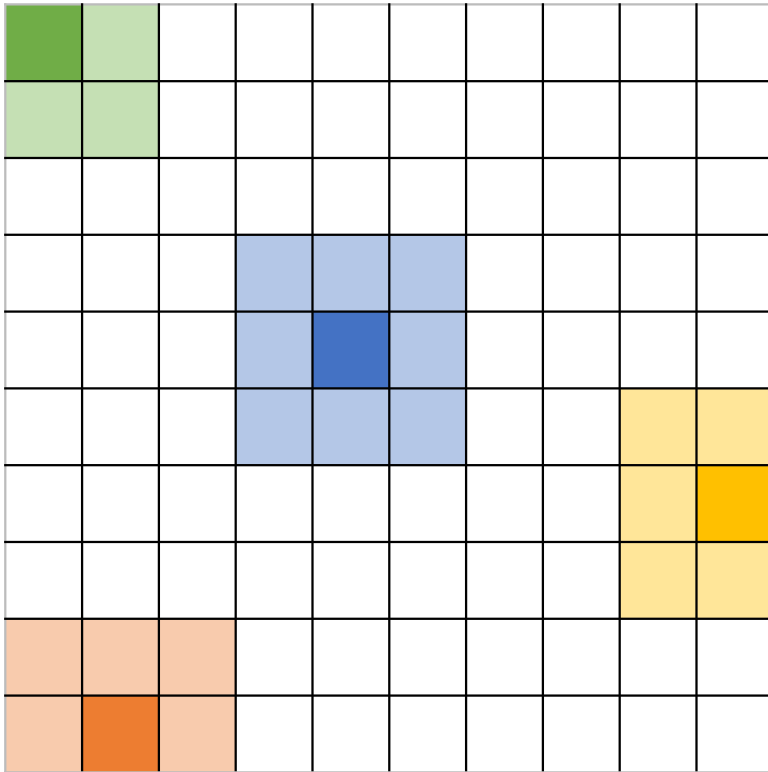
Parallelization approach: assign one thread to each *output* pixel, and have it read multiple *input* pixels


```
__global__ void blur_kernel(unsigned char* image, unsigned char* blurred,
                           unsigned int width, unsigned int height) {

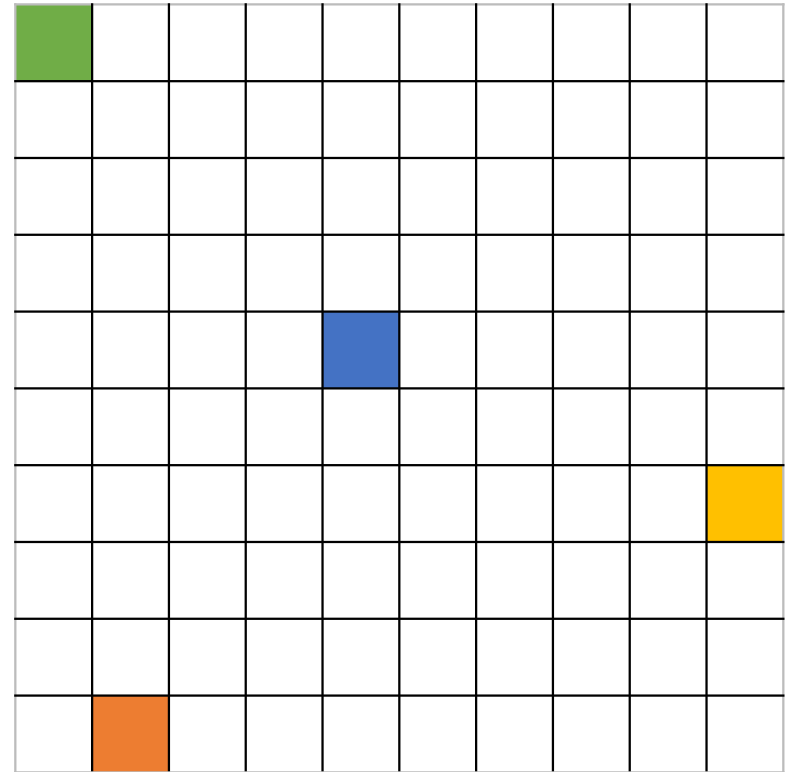
    int outRow = blockIdx.y*blockDim.y + threadIdx.y;
    int outCol = blockIdx.x*blockDim.x + threadIdx.x;

    if (outRow < height && outCol < width) {

        unsigned int average = 0;
        for(int inRow = outRow - BLUR_SIZE; inRow < outRow + BLUR_SIZE + 1; ++inRow) {
            for(int inCol = outCol - BLUR_SIZE; inCol < outCol + BLUR_SIZE + 1; ++inCol) {
                average += image[inRow*width + inCol];
            }
        }
        blurred[outRow*width + outCol] =
            (unsigned char)(average/((2*BLUR_SIZE + 1)*(2*BLUR_SIZE + 1)));
    }
}
```



Input Image



Output Blurred Image

Even if output pixel is in bounds, might be accessing out of bounds input pixel

```
__global__ void blur_kernel(unsigned char* image, unsigned char* blurred,
                           unsigned int width, unsigned int height) {

    int outRow = blockIdx.y*blockDim.y + threadIdx.y;
    int outCol = blockIdx.x*blockDim.x + threadIdx.x;

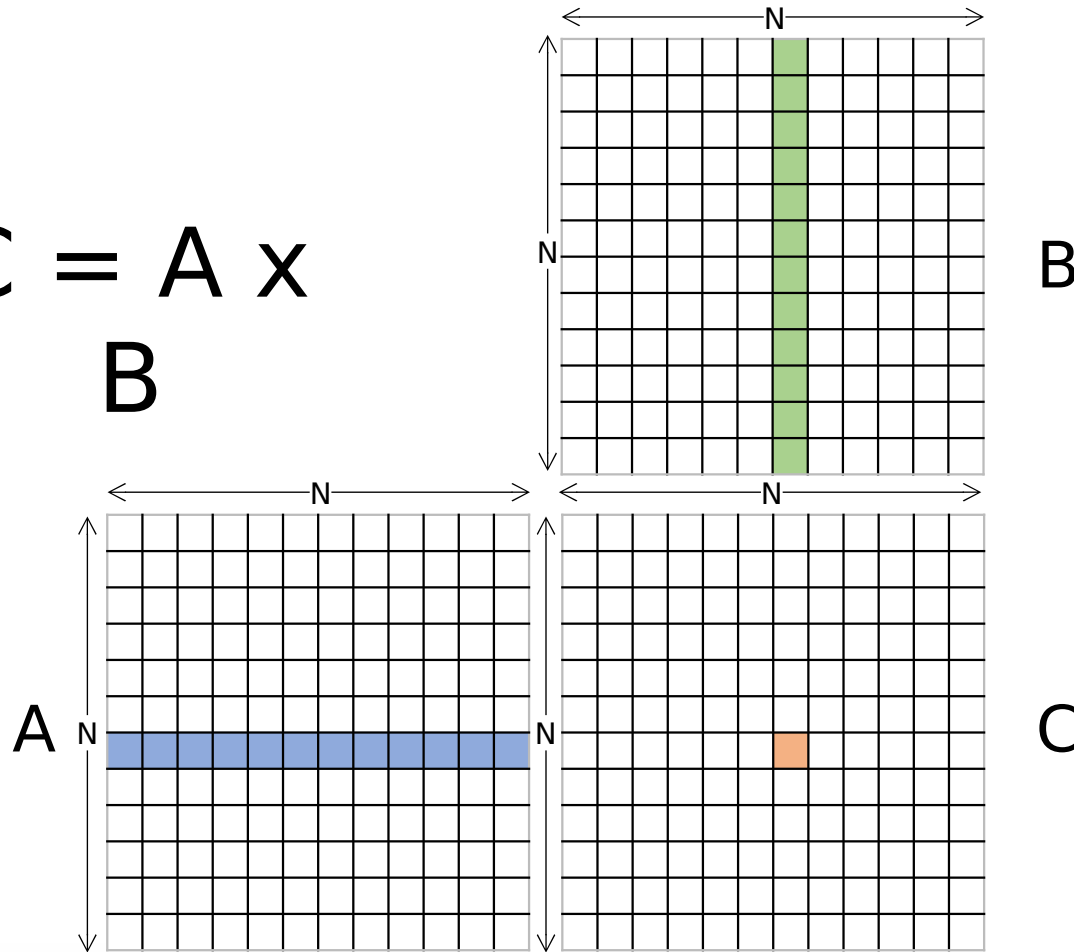
    if (outRow < height && outCol < width) {

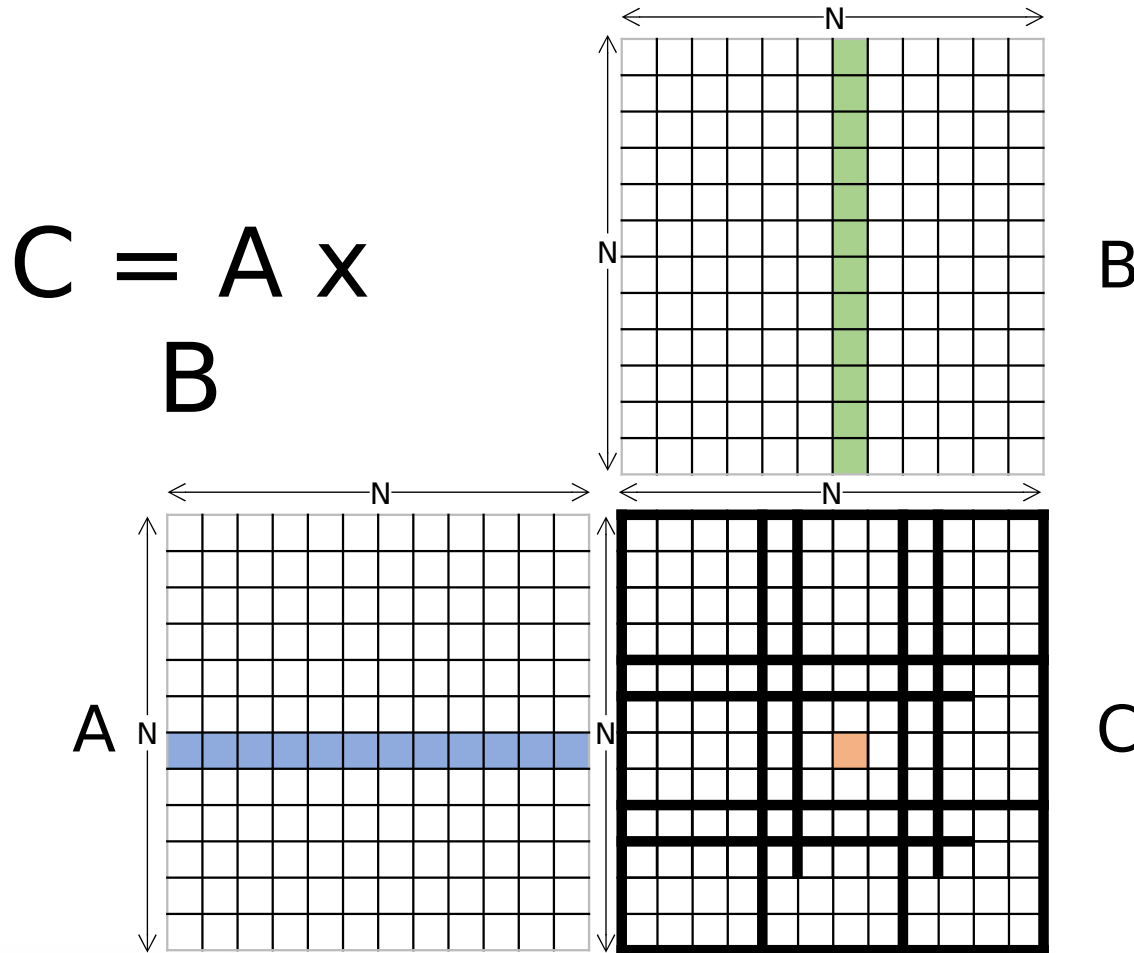
        unsigned int average = 0;
        for(int inRow = outRow - BLUR_SIZE; inRow < outRow + BLUR_SIZE + 1; ++inRow) {
            for(int inCol = outCol - BLUR_SIZE; inCol < outCol + BLUR_SIZE + 1; ++inCol) {
                if(inRow >= 0 && inRow < height && inCol >= 0 && inCol < width) {
                    average += image[inRow*width + inCol];
                }
            }
        }
        blurred[outRow*width + outCol] =
            (unsigned char)(average/((2*BLUR_SIZE + 1)*(2*BLUR_SIZE + 1)));
    }
}
```

boundary check

Rule of thumb: every memory access must have a corresponding guard that compares its indexes to the array dimensions

$$C = A \times B$$

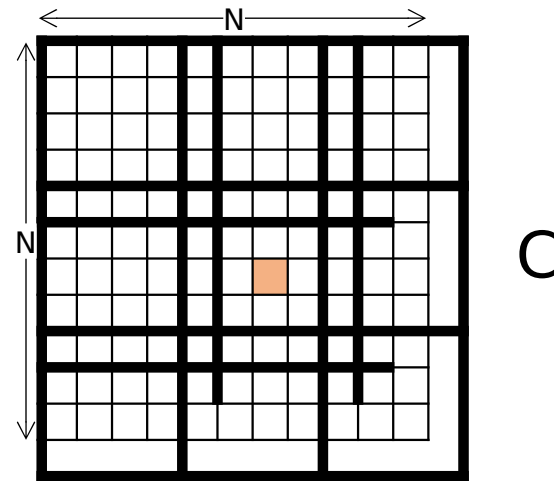
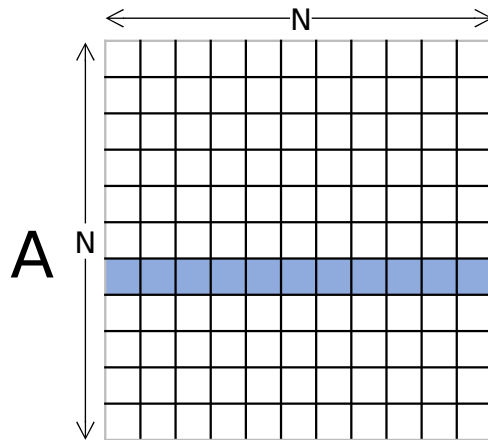
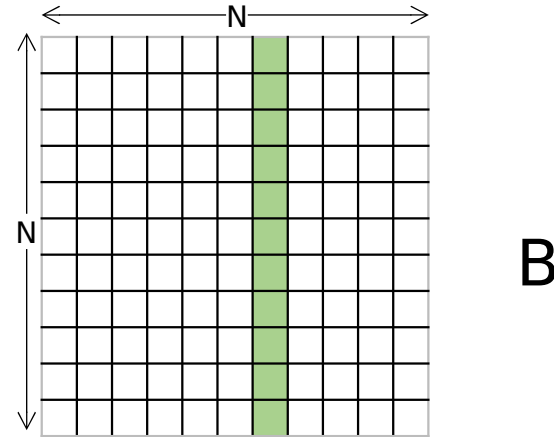




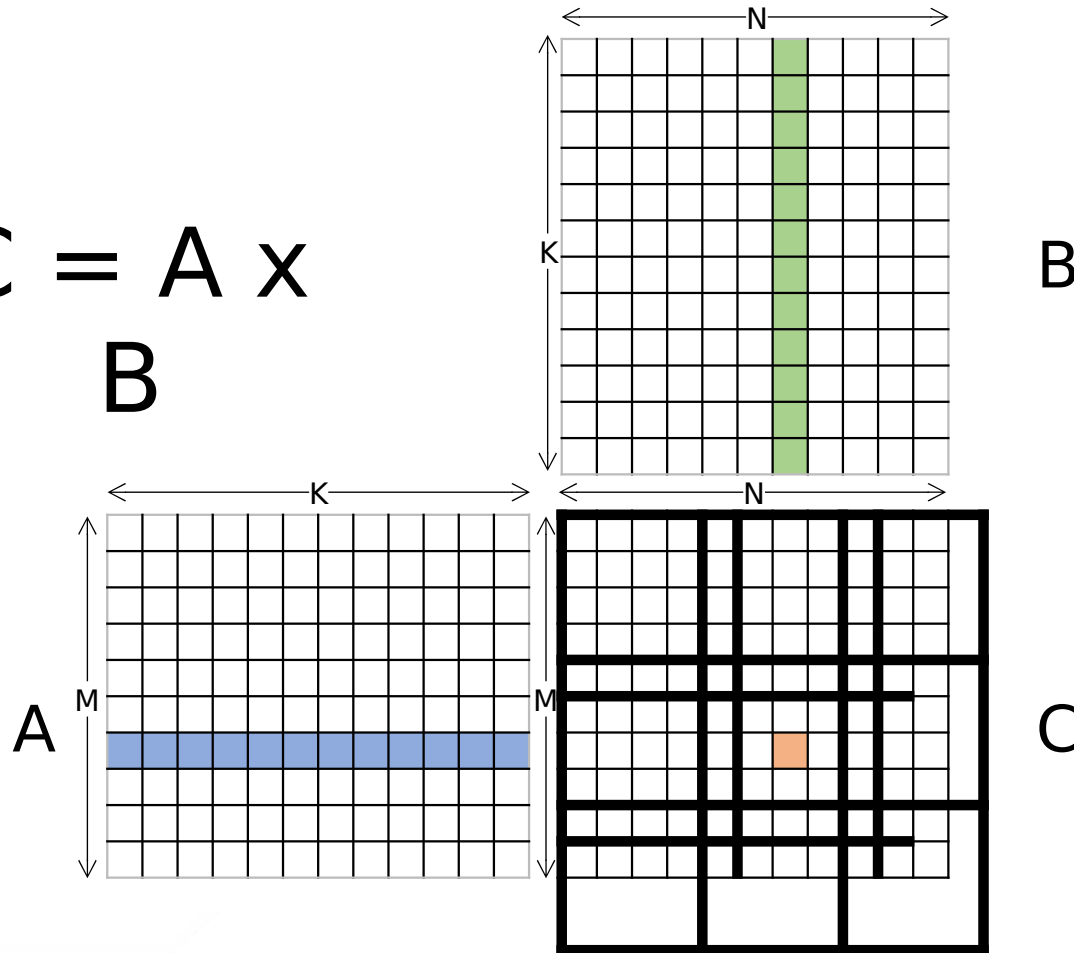
Parallelization approach: assign one thread to each element in the output matrix (C)

```
__global__ void mm_kernel(float* A, float* B, float* C, unsigned int N) {  
  
    unsigned int row = blockIdx.y*blockDim.y + threadIdx.y;  
    unsigned int col = blockIdx.x*blockDim.x + threadIdx.x;  
  
    float sum = 0.0f;  
    for(unsigned int i = 0; i < N; ++i) {  
        sum += A[row*N + i]*B[i*N + col];  
    }  
    C[row*N + col] = sum;  
  
}
```

$$C = A \times B$$



$$C = A \times B$$



- Wen-mei W. Hwu, David B. Kirk, and Izzat El Hajj. *Programming Massively Parallel Processors: A Hands-on Approach*. Morgan Kaufmann, 2022.